



COM 682

Cloud Native Development

*Hosted Web Endpoints (REST), APIs & Azure Data Storage
Platform*

CW1:

Design the REST API to support the creation, retrieval, updating, and deletion (CRUD) of various asset records using the Logic Apps.



Hosted Web Endpoints (REST)



Stateful and Stateless model



Stateful

1st Trip



2nd Trip

Stateless



What is REST?

REST stands for **R**epresentational **S**tate **T**ransfer

Architecture style for designing networked applications

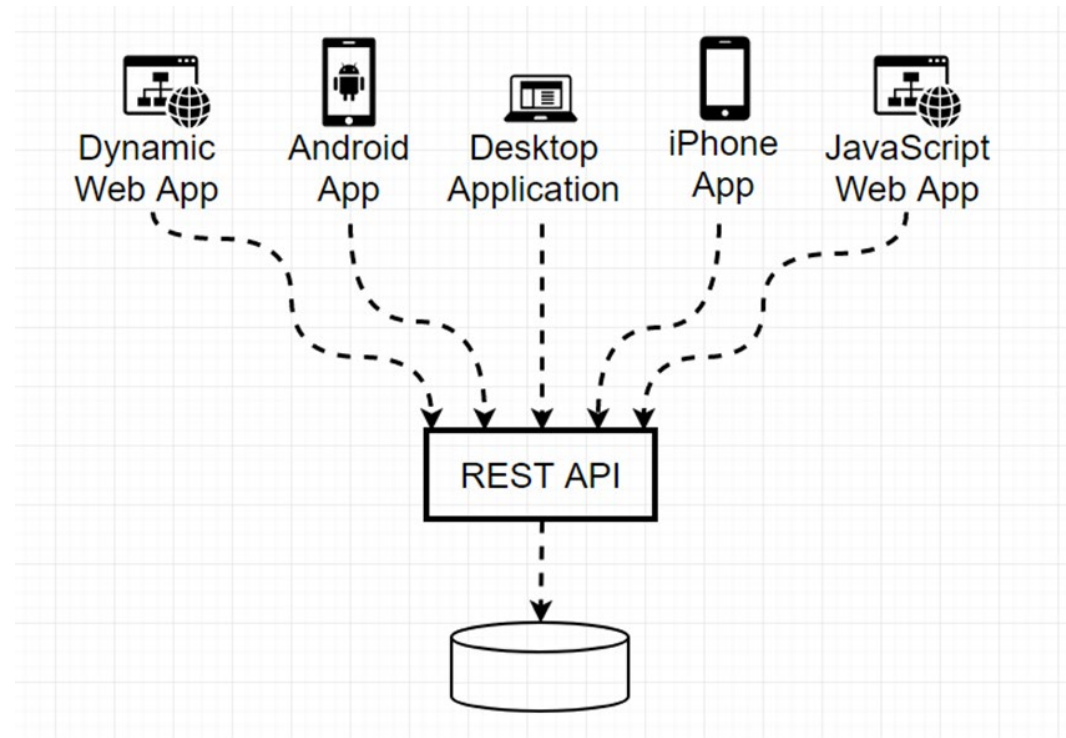
Relies on a **stateless**, **client-server** protocol, almost always **HTTP**

Treats server **objects as a resources** (data, file, feed etc). Resources can be created or destroyed.

Can be used virtually by any programming language

REpresentational **S**tate **T**ransfer (**REST**) has emerged as the common approach for communication within modern client-server solutions.

REST uses protocols that were originally made to serve and enable the World Wide Web. In particular, **REST uses concepts from the HTTP protocol.**



Stateless model

In 2000, Roy Fielding proposed Representational State Transfer (REST) as an architectural approach to design web services.

A primary advantage of REST over HTTP is that it **uses open standards** and does not bind the implementation of the API or the client applications to any specific implementation.

REST is an architectural style for building distributed systems based on hypermedia. REST is independent of any underlying protocol and is not necessarily tied to HTTP. However, most common REST API implementations use HTTP as the application protocol.

Here are some of the main design principles of **RESTful APIs** using HTTP:

- **REST APIs are designed around resources**, which are any kind of object, data, or service that can be accessed by the client.
- **A resource has an identifier**, which is a URI (Uniform Resource Identifier) that uniquely identifies that resource. For example, the URI for a particular customer order might be as shown below.

HTTP

 Copy

```
https://adventure-works.com/orders/1
```

Here are some of the **main design principles of RESTful APIs using HTTP**:

- **Clients interact with a service by exchanging *representations* of resources.** Many web APIs use JSON as the exchange format.

For example, a GET request to the URI (Uniform Resource Identifier) listed in last slide might return this response body.

JSON

 Copy

```
{"orderId":1,"orderValue":99.90,"productId":1,"quantity":1}
```

- REST APIs use a uniform interface, which helps to decouple the client and service implementations.

For REST APIs built on HTTP, the uniform interface includes using standard HTTP verbs to perform operations on resources. The most common operations are GET, POST, PUT (update a specific resource), PATCH (update partial resource), and DELETE.

GET	/pet/{petId}	Find pet by ID
PUT	/pet	Update an existing pet
DELETE	/pet/{petId}	Deletes a pet
POST	/pet/{petId}/uploadImage	uploads an image

GET retrieves a representation of the resource at the specified URI. The body of the response message contains the details of the requested resource.

POST creates a new resource at the specified URI. The body of the request message provides the details of the new resource. Note that POST can also be used to trigger operations that don't actually create resources.

PUT either creates or replaces the resource at the specified URI. The body of the request message specifies the resource to be created or updated.

PATCH performs a partial update of a resource. The request body specifies the set of changes to apply to the resource.

DELETE removes the resource at the specified URI.

⋮

EdgeX-testing / Send colour info to Application

API

PUT



http://192.168.1.28:5000/api/v1/device/edgexTutorial/changeColor

Web URI

Body



⋮

Resp

raw



JSON



Beautify

⋮

```
1 {  
2   "color": "red"  
3 }
```

REST APIs are driven by **hypermedia links** that are contained in the representation.

For example, the following shows a JSON representation of an order. It contains links to get or update the customer associated with the order.

JSON

 Copy

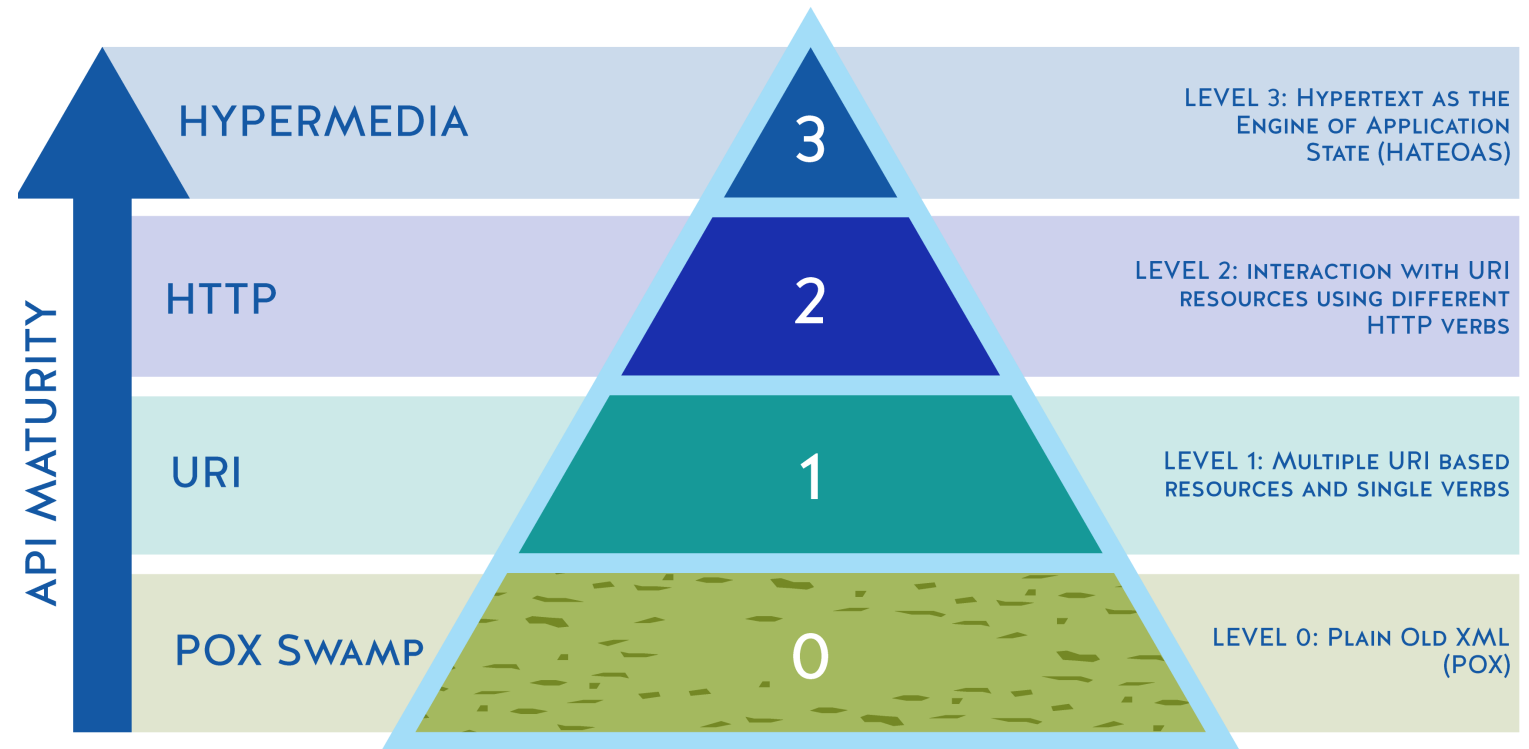
```
{
  "orderId":3,
  "productId":2,
  "quantity":4,
  "orderValue":16.60,
  "links": [
    {"rel":"product","href":"https://adventure-works.com/customers/3", "action":"GET" },
    {"rel":"product","href":"https://adventure-works.com/customers/3", "action":"PUT" }
  ]
}
```

REST basics

In 2008, Leonard Richardson proposed a **maturity model for web APIs**: (Uniform Resource Identifier)

RICHARDSON MATURITY MODEL

• **Level-3: HyperMedia/HATEOAS:**
'Most mature level that uses *HATEOAS (Hypertext As The Engine Of Application State)*. It's also known as HyperMedia, which basically consists of resource links and forms. Establishing connections between resources becomes easy as they don't require human intervention and aids client-driven automation.



What is API?

API (Application Programming Interface) is a set of rules that allows different software applications to talk to each other.

Think of a Restaurant:

- **You (The User)** want food.
- **The Menu (The API Contract)** tells you what you can order and how to order it.
- **The Waiter (The API)** takes your request (order) to the kitchen (backend) and brings back your food (response).
- **The Kitchen (Server/Logic Apps)** does the actual work—you don't see how it's done.



REST in Action: Our Multimedia Platform (CW1)

CRUD Operation	HTTP Method	Example Endpoint (The "Address")	What It Does
Create	POST	/api/assets	Uploads a new image/video
Retrieve	GET	/api/assets	Gets a list of all assets
Retrieve	GET	/api/assets/123	Gets details for asset #123
Update	PUT	/api/assets/123	Updates asset #123's metadata
Delete	DELETE	/api/assets/123	Deletes asset #123

‘Things to do setting up RESTful APIs’

a. Organize the API design around resources

By focusing on the business entities that the web API exposes.

When possible, **resource URIs should be based on nouns** (the resource) **and not verbs** (the operations on the resource).

HTTP

 Copy

`https://adventure-works.com/orders // Good`

`https://adventure-works.com/create-order // Avoid`

REST Organisation

A resource **doesn't have to be based on a single physical data item.**

For example, an order resource might be implemented internally as several tables in a relational database but presented to the client as a single entity.

b. Avoid creating APIs that simply mirror the internal structure of a database.

A client should not be exposed to the internal implementation. **(Security concern)**

c) Adopt a **consistent naming convention** in URIs.

In general, it helps to use plural nouns for URIs that reference collections.

It's a good practice to organize URIs for collections and items into a hierarchy. For example, /customers is the path to the customers collection, and /customers/5 is the path to the customer with ID equal to 5.

d) Consider the relationships between different types of resources and how you might expose these associations.

For example, the **/customers/5/orders** URI might represent all of the orders for customer 5.

You could also go in the other direction, and represent the association from an order back to a customer with a URI such as **/orders/99/customer**.

e) **avoid "chatty" web APIs** that expose a large number of small resources

Another factor is that all web requests impose a **load on the web server**.

The more requests, the bigger the load. Such an API may require a client application to send multiple requests to find all of the data that it requires.

Retrieving large objects can increase the latency of a request and incur additional bandwidth costs.

Google C

Google

Pro

Disco
availa
maps

Se

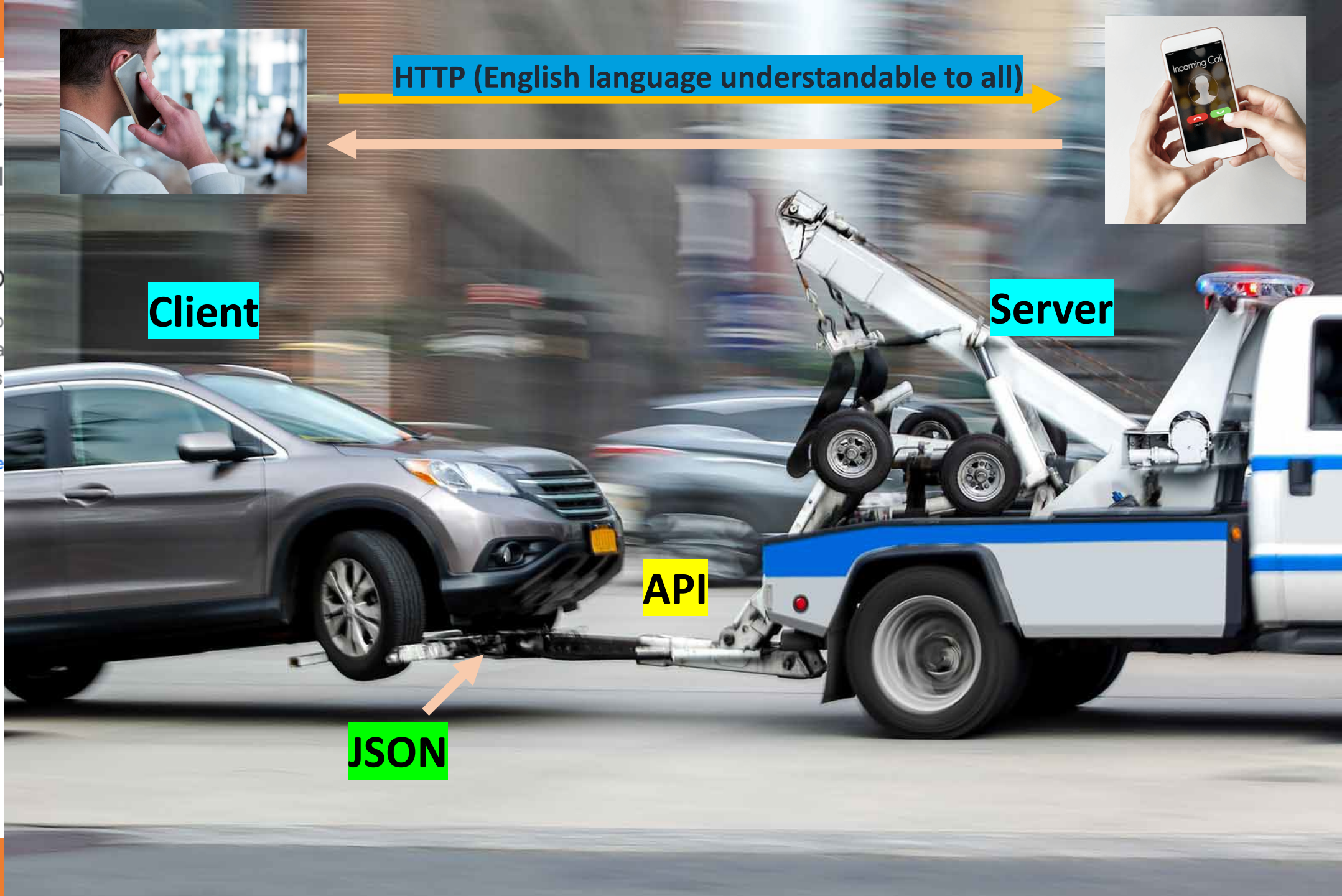


Client

HTTP (English language understandable to all)



Server



API

JSON

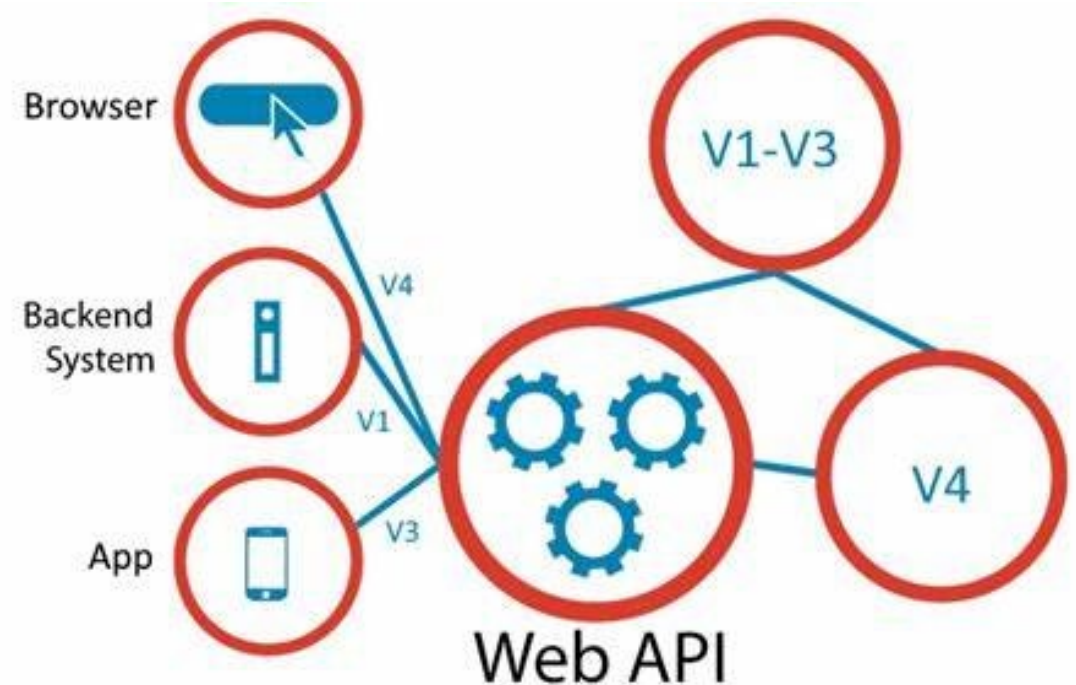
Versioning a RESTful web API

Versioning

It is highly unlikely that a web API will remain static.

As requirements change:

- new collections of resources may be added
- the relationships between resources might change
- the structure of the data in resources might be amended.





While updating a web API to handle new or differing requirements is a relatively straightforward process, you must consider the effects that such changes will have on client applications consuming the web API.



Versioning enables a web API to indicate the features and resources that it exposes, and a **client application can submit requests that are directed to a specific version of a feature or resource.**

3 types of Versioning

1) URI versioning

Each time you modify the web API or change the schema of resources, you add a version number to the URI for each resource. The previously existing URIs should continue to operate as before, returning resources that conform to their original schema.

An example of this is <https://adventure-works.com/v2/customers/3>

2) Query string versioning

Rather than providing multiple URIs, you can specify the version of the resource by using a parameter within the query string appended to the HTTP request, such as

<https://adventure-works.com/customers/3?version=2>.

3) Header versioning

Rather than appending the version number as a query string parameter, you could **implement a custom header that indicates the version of the resource.**

Version 1:

HTTP

 Copy

```
GET https://adventure-works.com/customers/3 HTTP/1.1  
Custom-Header: api-version=1
```

REST API how to be presented in UML diagram?

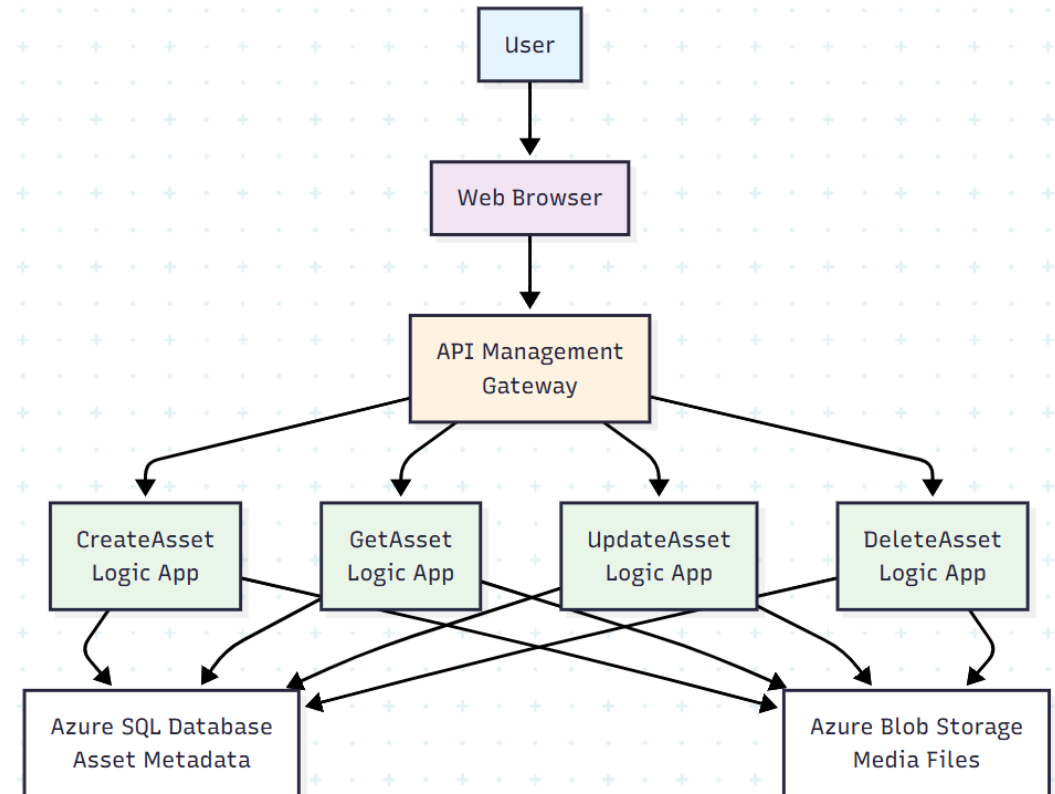
Step 1: Identify the Actors and Components

Every sequence diagram needs its cast of characters. For our cloud-native app, these are:

- The User:** The person using your web app.
- The Web Browser:** The user's interface.
- Azure API Management (APIM) or a single Logic App (HTTP Trigger):** This acts as your **API Gateway**. It's the public-facing endpoint that receives all requests. Using APIM is a more advanced, realistic pattern.
- Orchestrator Logic Apps:** These are the core "brain" of your API. You might have different ones for different operations (e.g., CreateAsset, GetAsset).
- Azure SQL Database:** For storing asset metadata (like title, description, owner).
- Azure Blob Storage:** For storing the actual multimedia file (the image, video, or sound).

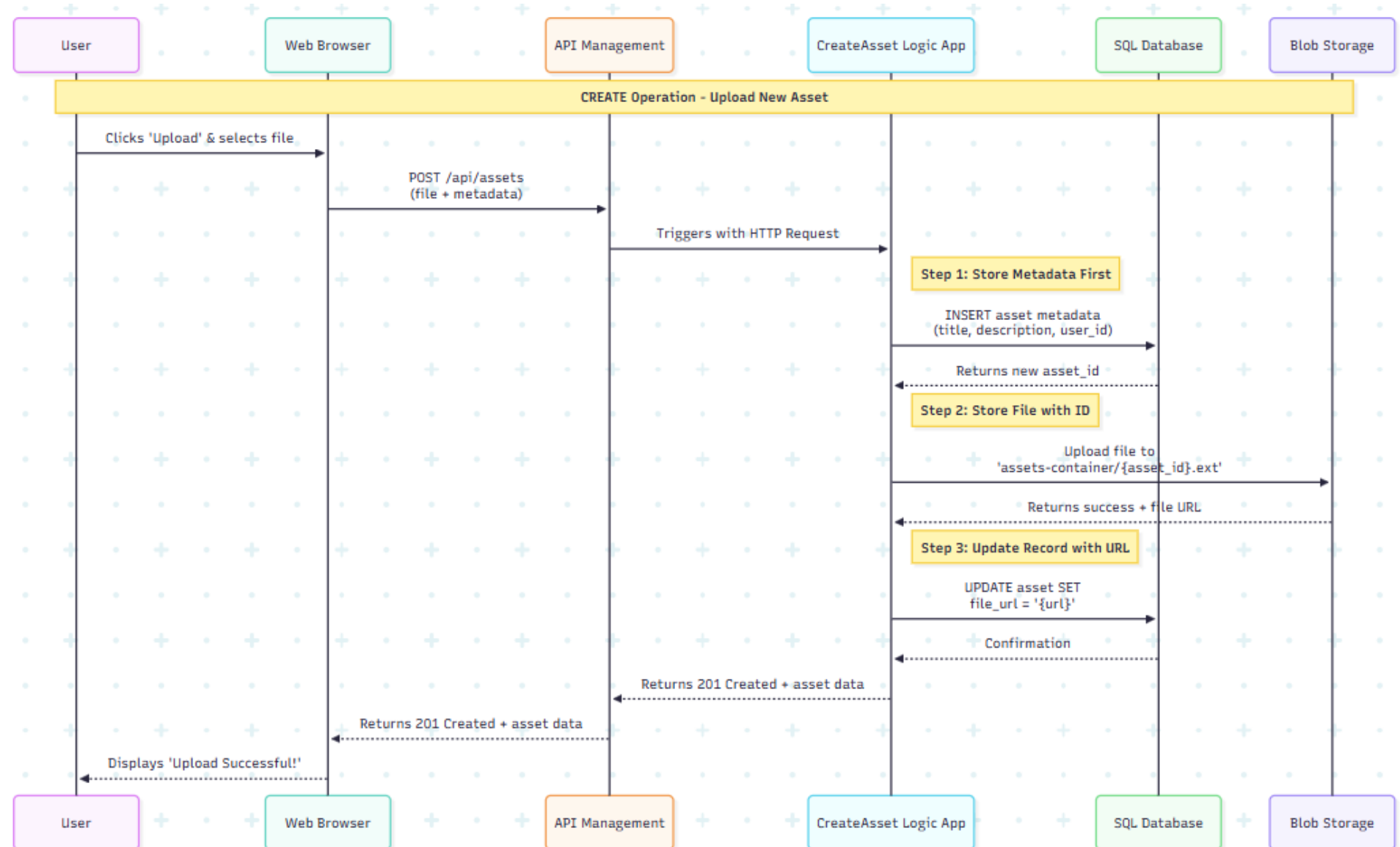
Step 2: Choose a Scenario and Draw the Lifelines

Let's take Creation, retrieval, updating, and deletion (CRUD) of an asset. We'll map this to a sequence diagram for Creating a New Asset."



UML diagrams

Step 3: Model the Message Flow for 'Create an Asset'



Step 4: Show How This Fulfills the Brief

The REST API Endpoint: POST /api/assets

The Use of Logic Apps: As the central orchestrator for the 'Create' operation.

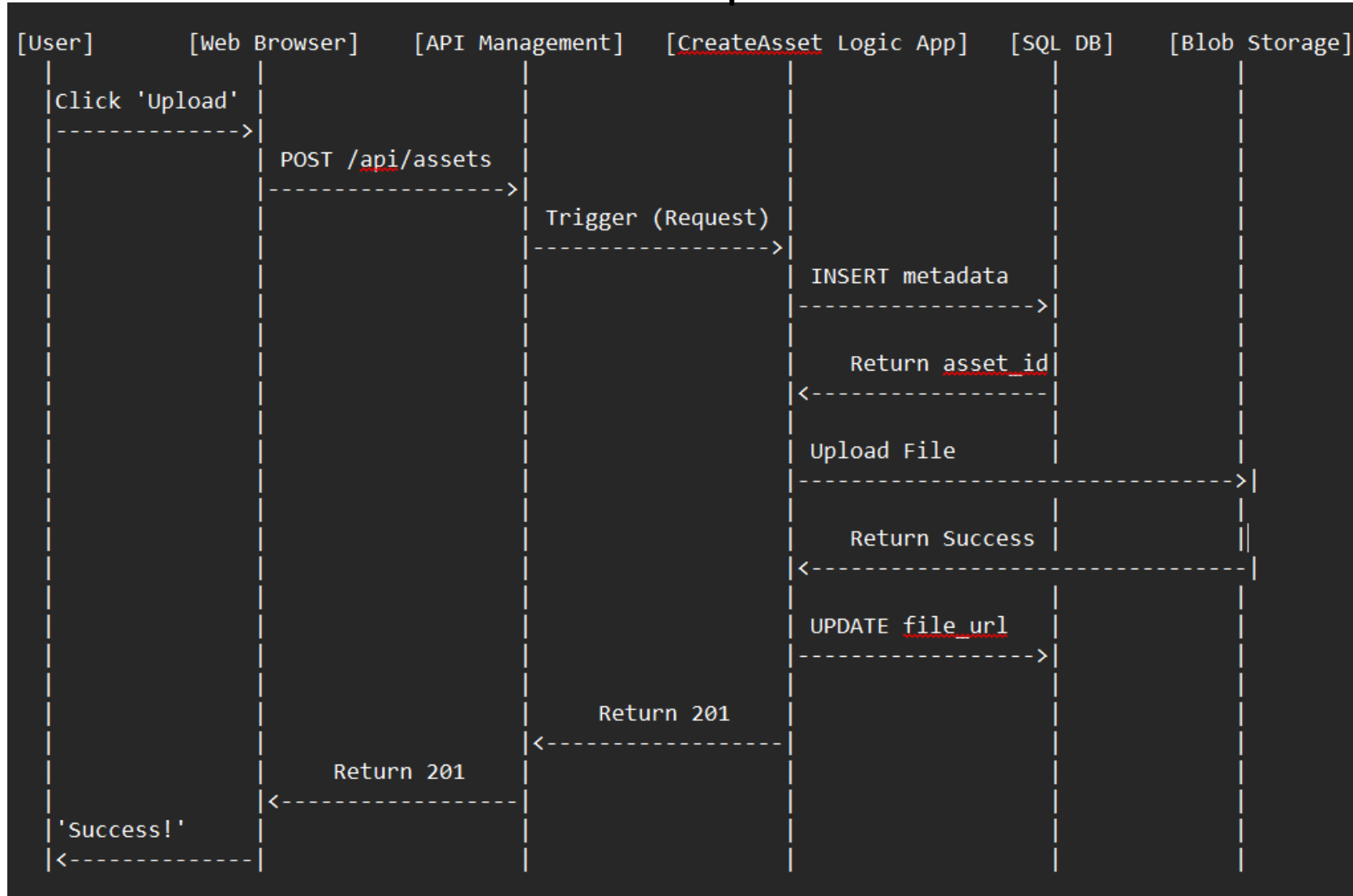
The CRUD Operation: 'Creation' (the 'C' in CRUD).

Integration with other Azure Services: SQL Database and Blob Storage.

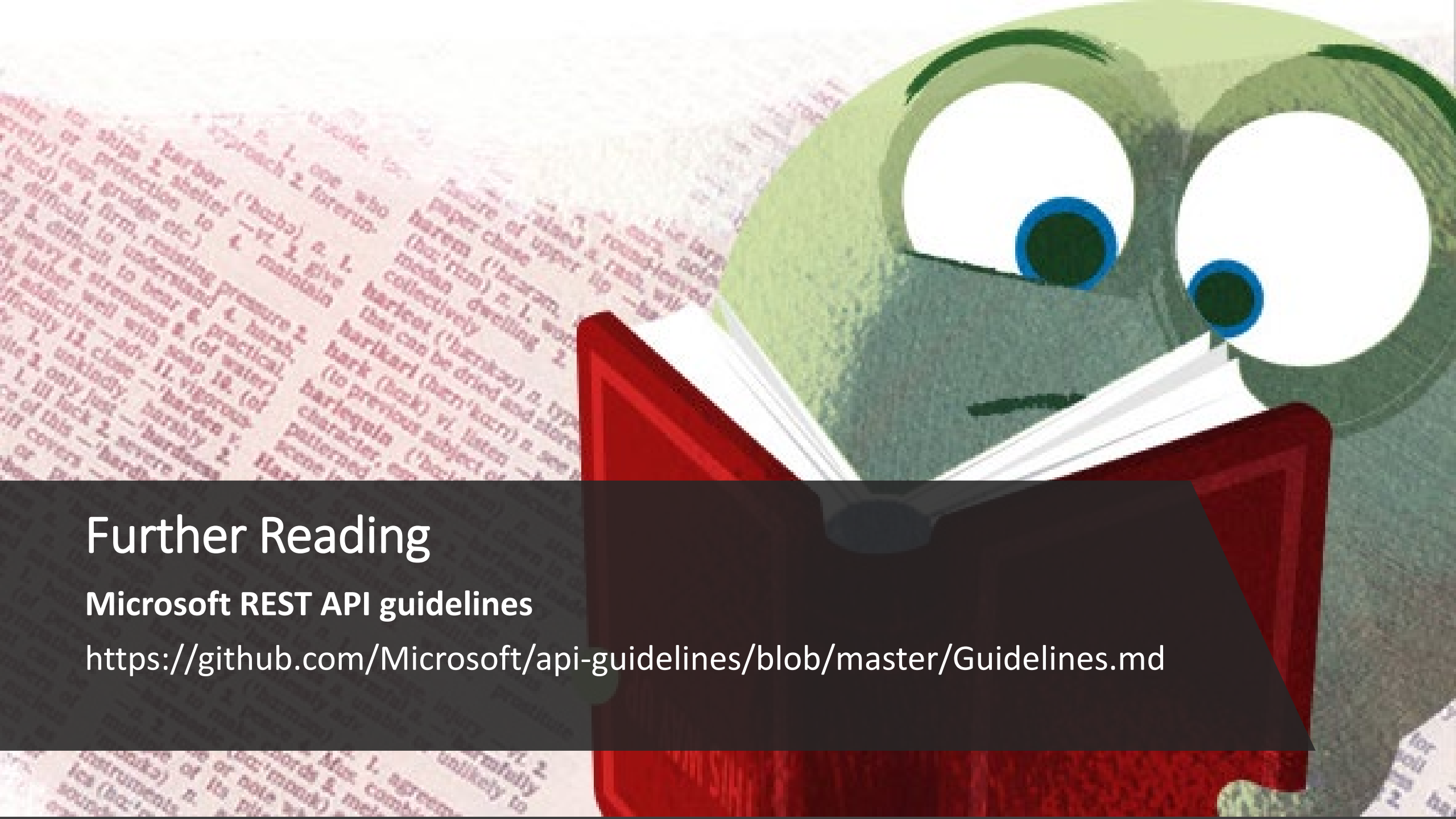
Key Takeaway: You are not drawing every single possible diagram. You are drawing **one or two representative diagrams** that prove you understand the pattern. A diagram for CREATE and another for GET are often enough to demonstrate your understanding of the entire CRUD lifecycle.

UML diagrams

For Example:



Key Takeaway: You are not drawing every single possible diagram. You are drawing **one or two representative diagrams** that prove you understand the pattern. A diagram for CREATE and another for GET are often enough to demonstrate your understanding of the entire CRUD lifecycle.



Further Reading

Microsoft REST API guidelines

<https://github.com/Microsoft/api-guidelines/blob/master/Guidelines.md>



Azure Data Storage Platform

Azure Data Storage platform

In the cloud, data can be stored in a variety of manners, dictated by needs and selection criteria.

The Azure Storage platform includes the following data services:

- **Azure Blobs:** A massively **scalable object store for text and binary data**. Also includes support for big data analytics through Data Lake Storage Gen2.
- **Azure Files:** **Managed file shares** for cloud or on-premises deployments.
- **Azure Queues:** A **messaging store for reliable messaging** between application components.
- **Azure Tables:** A NoSQL store for **schemaless storage of structured data**.
- **Azure Disks:** Block-level storage volumes for Azure VMs.

Blob Storage

Azure Blob storage is Microsoft's **object storage** solution for the cloud.

Blob storage is optimized for storing massive amounts of unstructured data. Unstructured data is data that doesn't adhere to a particular data model or definition, **such as text or binary data.**

Blob storage is designed for:

- Serving images or documents directly to a browser.
- Storing files for distributed access.
- Streaming video and audio.





Blob Storage

Blob storage is designed for:

- Writing to log files.
- Storing data for backup and restore, disaster recovery, and archiving.
- Storing data for analysis by an on-premises or Azure-hosted service.



Users or client applications can access objects in Blob storage via **HTTP/HTTPS**, from anywhere in the world.

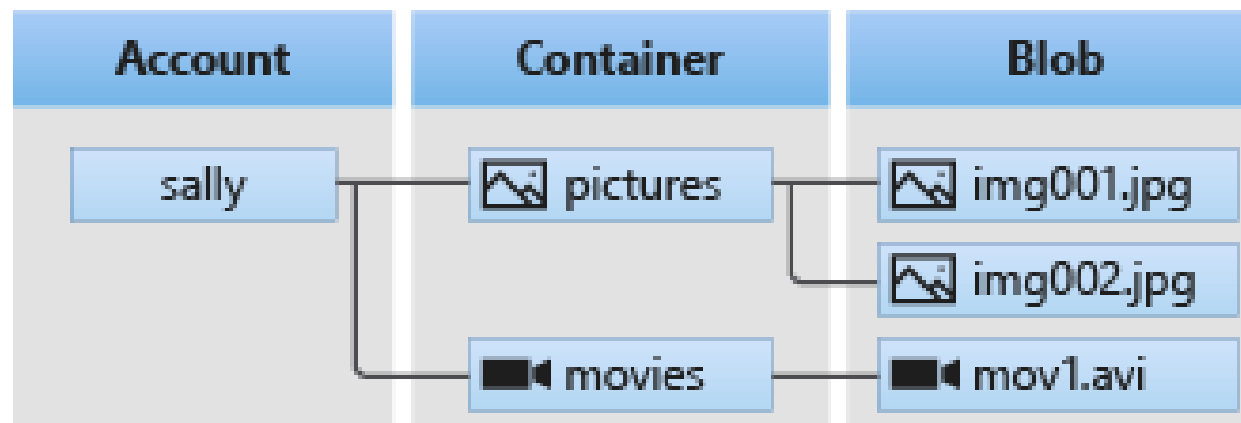
Objects in Blob storage are accessible via the **Azure Storage REST API, Azure PowerShell, Azure CLI, or an Azure Storage client library**. Client libraries are available for different languages, including: .NET, Java, Node.js, Python, Go, PHP and Ruby.

Blob storage resources

Blob storage consists of three types of resources:

- A. The storage account
- B. A container in the storage account
- C. A blob (object) in a container

The following diagram shows the relationship between these resources.



Storage accounts

A storage account **provides a unique namespace in Azure** for your data.

Every object that you store in Azure Storage **has an address that includes your unique account name.**

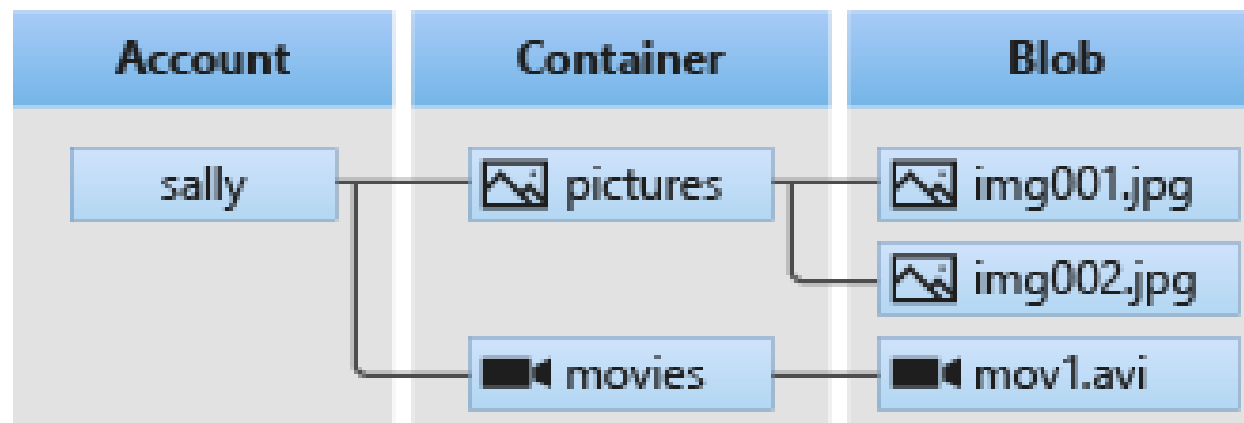
The combination of the account name and the Azure Storage blob endpoint forms the base address for the objects in your storage account.

For example, if your storage account is named `mystorageaccount`, then the default endpoint for Blob storage is:

```
http://mystorageaccount.blob.core.windows.net
```

Containers

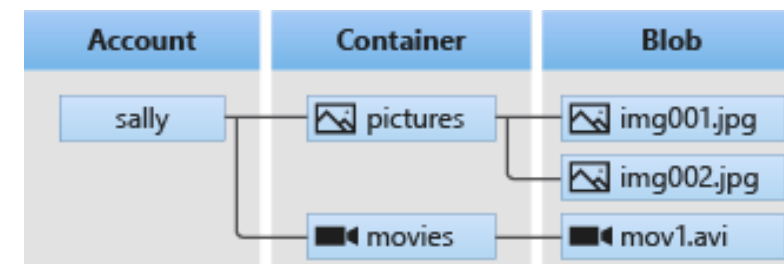
A container organizes a set of blobs, similar to a directory in a file system. A storage account can include an unlimited number of containers, and a container can store an unlimited number of blobs.



Blobs

Azure Storage supports three types of blobs:

- **Block blobs:** store text and binary data. Block blobs are made up of blocks of data that can be managed individually. Block blobs can store up to about 190.7 TiB.
- **Append blobs:** are made up of blocks like block blobs, but are optimized for append operations. Append blobs are ideal for scenarios such as logging data.
- **Page blobs:** store random access files up to 8 TiB in size. Page blobs store virtual hard drive (VHD) files and serve as disks for Azure virtual machines.



Blobs

Below is an example of a **RESTful HTTP call to Azure Blob Storage to create a container named *mycontainer*.**

The HTTP call includes authorization information for the client to access the bucket.

text

```
PUT https://myaccount.blob.core.windows.net/mycontainer?restype=container HTTP/1.1
```

Request Headers:

```
x-ms-version: 2011-08-18
```

```
x-ms-date: Sun, 25 Sep 2011 22:50:32 GMT
```

```
x-ms-meta-Name: StorageSample
```

```
Authorization: SharedKey myaccount:Z5043vY9MesKNh0PntksNc9nbXSSqGHueE00JdjidoQ=
```

Blobs

Blob Storage can process the request and will send back an HTTP response similar to the following:

text

Response Status:

HTTP/1.1 201 Created

Response Headers:

Transfer-Encoding: chunked

Date: Sun, 25 Sep 2011 23:00:12 GMT

ETag: "0x8CB14C3E29B7E82"

Last-Modified: Sun, 25 Sep 2011 23:00:06 GMT

x-ms-version: 2011-08-18

Server: Windows-Azure-Blob/1.0 Microsoft-HTTPAPI/2.0

In the response, Azure has acknowledged the request, indicated that the request was successful (with a "201 Created" message), and returned some information regarding the request.

```
for object to mirror_mod.mirror_object
operation == "MIRROR_X":
mirror_mod.use_x = True
mirror_mod.use_y = False
mirror_mod.use_z = False
operation == "MIRROR_Y":
mirror_mod.use_x = False
mirror_mod.use_y = True
mirror_mod.use_z = False
operation == "MIRROR_Z":
mirror_mod.use_x = False
mirror_mod.use_y = False
mirror_mod.use_z = True
```

```
@selection at the end -add
mirror_ob.select= 1
mirror_ob.select=1
context.scene.objects.remove(mirror_ob)
("Selected" + str(modifier.name))
mirror_ob.select = 0
= bpy.context.selected_objects
data.objects[one.name].select
print("please select exactly one object")
```

--- OPERATOR CLASSES ---

```
bpy.types.Operator):
X mirror to the selected
object.mirror_mirror_x"
mirror X"
```

Azure Files

Azure Files enables you to set up highly available network file shares that can be accessed by using the standard Server Message Block (SMB) protocol.

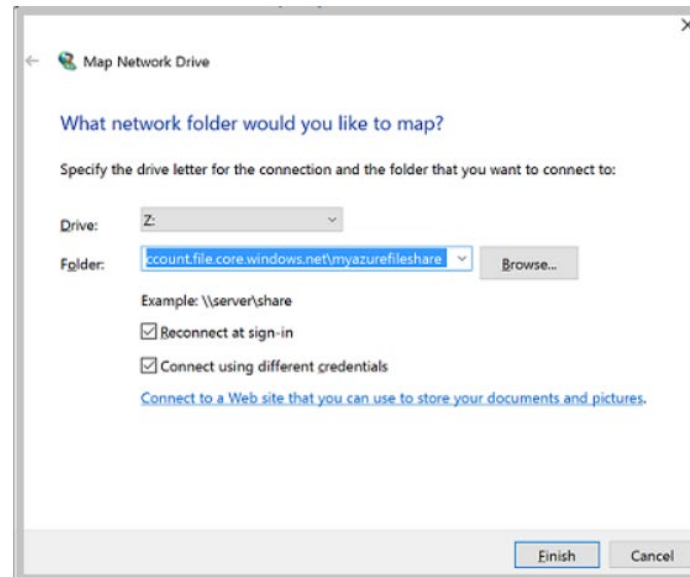
That means that multiple VMs can share the same files with both read and write access. You can also read the files using the REST interface or the storage client libraries.

One thing that distinguishes Azure Files from files on a corporate file share is that you can access the files from anywhere in the world using a URL that points to the file and includes a shared access signature (SAS) token.

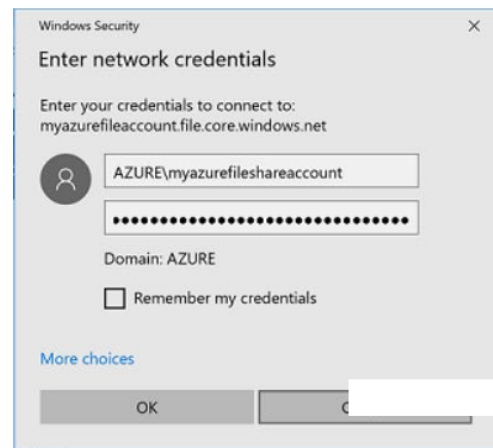
You can generate SAS tokens; they allow specific access to a private asset for a specific amount of time.

File shares can be used for many common scenarios:

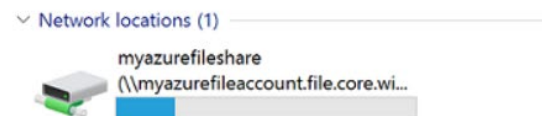
- **Many on-premises applications use file shares.** This feature makes it easier to migrate those applications that share data to Azure. If you mount the file share to the same drive letter that the on-premises application uses, the part of your application that accesses the file share should work with minimal, if any, changes.
- **Configuration files can be stored on a file share and accessed from multiple VMs.** Tools and utilities used by multiple developers in a group can be stored on a file share, ensuring that everybody can find them, and that they use the same version.
- **Resource logs, metrics, and crash dumps** are just three examples of data that can be written to a file share and processed or analyzed later.



4. Use the storage account name prepended with `AZURE\` as the username and a storage account key as the password.



5. Use Azure file share as desired.



Azure Queues



Azure Queues

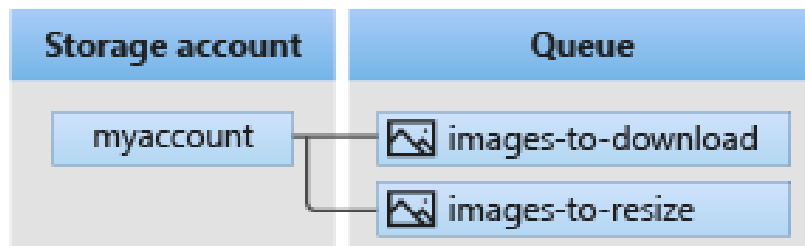
- The Azure Queue service is **used to store and retrieve messages.**
- Queue messages can be up to 64 KB in size, and a queue can contain millions of messages. Queues are generally used to store lists of messages to be processed asynchronously.

Azure Queues

- **Queue storage**
- For example, say you want your customers to be able to upload pictures, and you want to create thumbnails for each picture.
- You could have your customer wait for you to create the thumbnails while uploading the pictures.
- An alternative would be to use a queue:
 - When the customer finishes their upload, **write a message** to the queue.
 - Have an Azure Function retrieve **the message** from the queue and create the thumbnails.
- Each of the parts of this processing can be scaled separately, giving you more control when tuning it for your usage.



Azure Queues



- **Queue Storage concepts**
- Queue Storage contains the following components:
 - **Storage account:** All access to Azure Storage is done through a storage account. This has permissions related concerns.
 - **Queue:** A queue contains a set of messages. The queue name must be all lowercase. For information on naming queues, see Naming queues and metadata.

Table storage

Azure Table storage is a **service that stores non-relational structured data** (also known as structured NoSQL data) in the cloud, providing a key/attribute store with a **schemaless** design.

Because **Table storage is schemaless**, it's easy to adapt your data as the needs of your application evolve.

Access to Table storage data is fast and cost-effective for many types of applications, and is typically lower in cost than traditional SQL for similar volumes of data.

You can use Table storage to store flexible datasets like user data for web applications, address books, device information, or other types of metadata your service requires. You can store any number of entities in a table, and a storage account may contain any number of tables, up to the capacity limit of the storage account.

Table storage

- Azure Table storage stores large amounts of structured data. The service is a NoSQL datastore which accepts authenticated calls from inside and outside the Azure cloud. Azure tables are ideal for storing structured, non-relational data.
- Common uses of Table storage include:
 - Storing TBs of structured data capable of serving web scale applications
 - Storing datasets that **don't** require complex joins, foreign keys, or stored procedures and can be denormalized for fast access
 - Quickly querying data using a clustered index
 - You can use Table storage to store and query huge sets of structured, non-relational data, and your tables will scale as demand increases.



Tables store data as collections of entities. Entities are similar to rows.

An entity has a primary key and a set of properties. A property is a name, typed-value pair, similar to a column.

The Table service does not **enforce any schema for tables**, so two entities in the same table may have different sets of properties. Developers may choose to enforce a schema on the client side. A table may contain any number of entities.

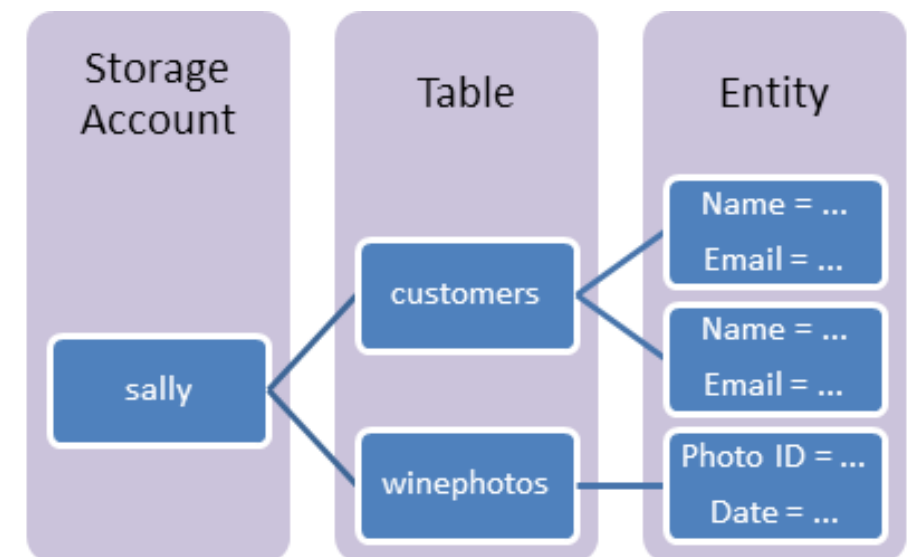


Table storage contains the following components:

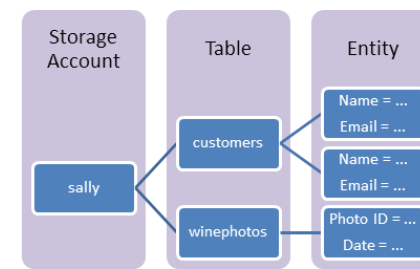
- **URL:** A URL to access, retrieve, manipulate data.

Azure Table Storage accounts use:

format: `http://<storage account>.table.core.windows.net/<table>`

Azure Cosmos DB Table API accounts use:
`http://<storage account>.table.cosmosdb.azure.com/<table>`

- **Accounts:** All access to Azure Storage is done through a storage account. For more information about storage accounts, see Storage account overview.





Disk storage

An **Azure managed disk** is a virtual hard disk (VHD). You can think of it like a physical disk in an on-premises server but, virtualized.

Azure-managed disks are stored as page blobs, which are a random IO storage object in Azure. We call a managed disk 'managed' because it is an abstraction over page blobs, blob containers, and Azure storage accounts.

With managed disks, all you have to do is provision the disk, and Azure takes care of the rest.

Benefits of managed disks

Highly durable and available

Managed disks are designed for 99.999% availability. Managed disks achieve this by providing you with three replicas of your data, allowing for high durability. If one or even two replicas experience issues, the remaining replicas help ensure persistence of your data and high tolerance against failures.

Integration with availability sets

Managed disks are integrated with availability sets to ensure that the disks of VMs in an availability set are sufficiently isolated from each other to avoid a single point of failure. Disks are automatically placed in different storage scale units (stamps). If a stamp fails due to hardware or software failure, only the VM instances with disks on those stamps fail.

Benefits of managed disks

Azure Backup support

To protect against regional disasters, Azure Backup can be used to create a backup job with time-based backups and backup retention policies. This allows you to perform VM or managed disk restorations at will. Currently Azure Backup supports disk sizes up to 32 terabyte (TiB) disks. Learn more about Azure VM backup support.

Benefits of managed disks

Download/Upload your VHD

Direct download/upload makes it easy to transfer your vhd to an Azure managed disk. It enables easier migration to the cloud and vice versa.

Encryption

Managed disks offer two different kinds of encryption. The first is Server Side Encryption (SSE), which is performed by the storage service. The second one is Azure Disk Encryption (ADE), which you can enable on the OS and data disks for your VMs



Any Questions?